

Computer Architecture 2025-26

Practical Assignment for Week 4

March 18, 2026

Introduction

In Week 1, you implemented a general-purpose register using Java. In this assignment, you will revisit the same problem, but using the C programming language.

The objective is to help you transition from Java to C by translating object-oriented concepts (such as classes and methods) into procedural C constructs (such as arrays and functions). You are encouraged to reuse your Java solution as a reference and progressively adapt it to C.

Note: In this assignment, you must not use an IDE. All files should be edited using a text editor, and compilation and execution must be performed from the command line.

General Purpose Register – C Version

As in the Java version, a register is modelled as an array of boolean values. In C, we define this structure using the `typedef` construct:

```
#define GP_REGISTER_SIZE 8

typedef bool gp_register[GP_REGISTER_SIZE];
```

This defines a new type `gp_register`, corresponding to an array of 8 boolean values.

Functions to Implement

You must implement the following functions, which correspond closely to the methods developed in the Java version:

```
void gp_register_set_bit(gp_register reg, int index, bool value);
int gp_register_get_bit(const gp_register reg, int index);
void gp_register_set(gp_register reg, int value);
unsigned int gp_register_get_unsigned(const gp_register reg);
int gp_register_get_int(const gp_register reg);
```

These functions operate directly on the register passed as argument. Remember that in C, the name of an array corresponds to the address of its first element. Therefore, when passing an array to a function, you are effectively passing a reference to the beginning of the register, and any modifications made inside the function will affect the original register.

C does not provide built-in support for exceptions. Therefore, we suggest using `assert` to ensure that indices remain within valid bounds and that values are within the representable range. As you may read in the online documentation (<https://man7.org/linux/man-pages/man3/assert.3.html>), to use the `assert` function, you must include header file `assert.h`.

Files and Structure

The function prototypes are provided in header file `gp_register.h`. Your task is to implement these functions in source file `gp_register.c`.

To assist you in testing your implementation, we provide file `gp_register_main.c`, which contains a `main` function that applies functions `gp_register_set`, `gp_register_get_unsigned` and `gp_register_get_int` to a number read from the command line.

Compilation

To compile your program use the C language compiler (`cc` or, commonly, `gcc`):

```
cc -Wall -o gp_register gp_register_main.c gp_register.c
```

where:

- `-Wall` enables most compiler warnings (recommended)
- `-o gp_register` specifies the name of the executable (program) to be generated from the source files. In this example, the executable is named `gp_register`, but you may choose any name you prefer.

Execution

Run the program using the command below. Note that you will typically need to include `./` to indicate that the executable is located in the current directory (folder):

```
./gp_register number_to_test
```

Examples of application are:

```
$ ./gp_register 5
register: 0000 0101
unsigned: 5      signed: 5
```

```
$ ./gp_register 200
register: 1100 1000
unsigned: 200    signed: -56
```

```
$ ./gp_register -2
register: 1111 1110
unsigned: 254    signed: -2
```